# Broker | Guia técnico da API

### 02/11/2026 | 37 operações públicas | GET, POST, PUT e DELETE

Referência para integrar um backend à API HTTPS do Broker: conexões QR, texto, mídia privada, eventos e controle Chatwoot opcional. Os exemplos usam somente IDs sintéticos e variáveis de credencial.

## Acesso e convenções

```
BASE_URL="https://broker.example"
# Use a origem e a chave recebidas para sua instalação.
# BROKER_API_KEY é uma variável privada do backend.
curl "$BASE_URL/v1/provider-accounts?provider=BAILEYS&limit=100" \
  -H "X-JRC-API-Key: $BROKER_API_KEY"
```

A chave define o tenant; não envie organizationId para selecionar outra empresa. Use uma única credencial por requisição. Guarde a chave no backend. A documentação interativa está em /documentation/; Execute realiza a operação solicitada quando a escrita está habilitada.

| Escopos genéricos | Permissão |
|---|---|
| instances:read / instances:write | Consultar / administrar conexões. |
| messaging:read / messaging:send | Consultar canais, conversas e anexos / enviar texto. |
| media:upload / media:send | Carregar e consultar arquivo / enviar referência com messaging:send. |
| webhooks:manage | Gerir receptores e consultar eventos/entregas. |

## Criar, parear e ativar um canal QR

```
POST /v1/instances
{"name":"Canal de teste","provider":"BAILEYS",
  "providerAccountId":"11111111-1111-4111-8111-111111111111"}

POST /v1/instances/{id}/connect
{}
GET /v1/instances/{id}/status
POST /v1/messaging/instances/{id}/partner-activate
{}
```

Nos POST JSON: Content-Type: application/json e Idempotency-Key. Troque os UUIDs sintéticos pelos IDs retornados. Obtenha providerAccountId na primeira consulta; guarde instance.id. Exiba action QR_CODE/PAIRING_CODE apenas ao operador autorizado e aguarde CONNECTED. A ativação retorna o canal plano: response.id é channelId. Não ativa bot.

## Enviar texto individual

```
POST /v1/messaging/channels/{id}/partner-text
{"conversationId":"11111111-1111-4111-8111-111111111111","text":"Mensagem de teste"}
```

Texto: 1 a 4096 caracteres. conversationId ou phoneNumber E.164, nunca ambos. HTTP 202 admite na fila; acompanhe as mensagens e os eventos para SENT, DELIVERED, READ ou falha. Uma chave idempotente representa a mesma intenção, inclusive após timeout.

# Mídia privada | carregar e enviar

Primeiro recorte: envio individual QR, provider BAILEYS e BROKER_TRANSPORT. Imagem, áudio, vídeo e documento; sem grupos, templates Meta, stickers, PTT ou chamadas neste perfil. Áudio aceito pelo contrato ainda exige validar codec/reprodução no sistema consumidor.

## 1. POST binário - scope media:upload

```
curl -X POST "$BASE_URL/v1/messaging/channels/$CHANNEL_ID/media" \
  -H "X-JRC-API-Key: $BROKER_API_KEY" \
  -H "Idempotency-Key: 22222222-2222-4222-8222-222222222222" \
  -H "Content-Type: application/octet-stream" \
  -H "X-JRC-Media-Mime: application/pdf" \
  -H "X-JRC-Media-Kind: document" \
  -H "X-JRC-Media-Name: arquivo.pdf" \
  --data-binary @arquivo.pdf
```

X-JRC-Media-Name usa encodeURIComponent(nome). Envie bytes, sem base64, multipart ou URL remota. Limite global: 1 byte a 16 MiB; imagem até 5 MiB. MIME, assinatura dos bytes, kind e nome precisam ser compatíveis. O armazenamento é privado e sujeito à quota do tenant.

```
HTTP 201 - exemplo de metadados
{
  "id": "11111111-1111-4111-8111-111111111111",
  "channelId": "11111111-1111-4111-8111-111111111111",
  "status": "READY",
  "kind": "document",
  "fileName": "arquivo.pdf",
  "mimeType": "application/pdf",
  "byteSize": 4096,
  "sha256": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"
}
```

## 2. GET metadados - reconciliar o estado

```
GET /v1/messaging/channels/{id}/media/{mediaId}
X-JRC-API-Key: $BROKER_API_KEY
```

201 cria o recurso durável; não comprova bytes prontos nem envio. READY permite envio; STORING exige consulta/reconciliação; FAILED informa falha. GET exige media:upload e não refaz upload nem envia mensagens. Replay com mesma chave UUID e mesmos bytes/metadados retorna a mesma referência; alteração retorna 409. Não repita um PUT incerto com nova intenção.

## 3. POST referência - messaging:send + media:send

```
POST /v1/messaging/channels/{id}/partner-media
Content-Type: application/json
Idempotency-Key: 33333333-3333-4333-8333-333333333333

{"conversationId":"11111111-1111-4111-8111-111111111111",
  "mediaId":"11111111-1111-4111-8111-111111111111","caption":"Arquivo de teste"}
```

Use mediaId de arquivo READY do mesmo tenant/canal. Caption é opcional, até 1024 caracteres. Nome, MIME e kind vêm do registro validado. HTTP 202 é fila; upload não implica envio. Chaves antigas não ganham os scopes novos. Revogação, expiração ou perda de scope impedem despacho pendente; um envio físico já efetuado não pode ser recolhido.

# Eventos | receber, confirmar e retomar

## POST - cadastrar receptor HTTPS

```
POST /v1/partner-webhooks
Content-Type: application/json
Idempotency-Key: receptor-teste-01

{"url":"https://backend.example/hooks/broker"}
```

Exige webhooks:manage. Guarde endpoint.id e signingSecret retornados. O segredo HMAC é separado da API key e não aparece nas listagens. HTTPS público, porta 443, DNS/certificado válidos, sem query, fragmento ou credenciais; até cinco endpoints ativos por tenant.

ackMode é opcional: HTTP mantém o aceite de transporte 2xx, inclusive 204 sem corpo. Para exigir recibo de persistência, cadastrar com ackMode=PERSISTENCE_RECEIPT_V1. Mesma chave idempotente exige URL normalizada e modo iguais; divergência retorna 409. URL e modo são imutáveis.

| Evento | Conteúdo observado |
|---|---|
| message.received | Mensagem recebida. |
| message.created | Registro de saída, inclusive origem observada. |
| message.status | Estado de mensagem. |
| connection.status | Estado de conexão QR ou Meta. |

## Verificar assinatura e confirmar recebimento

```
X-JRC-Event-Id: UUID_DO_EVENTO
X-JRC-Delivery-Id: UUID_DA_ENTREGA
X-JRC-Endpoint-Id: UUID_DO_ENDPOINT
X-JRC-Ack-Mode: HTTP ou PERSISTENCE_RECEIPT_V1
X-JRC-Timestamp: UNIX_SEGUNDOS
X-JRC-Signature: sha256=HEX_HMAC

HEX_HMAC = HMAC_SHA256(signingSecret,
  timestamp + "." + eventId + "." + exactRawBody)
```

Valide os bytes brutos antes do JSON, assinatura em tempo constante, eventId do header contra o envelope e timestamp em janela de cinco minutos. Persista eventId único antes do 2xx. O transporte pode se repetir e chegar fora de ordem: deduplique e preserve a origem para evitar loop de reenvio.

No modo PERSISTENCE_RECEIPT_V1, responder somente após COMMIT com 2xx, Content-Type application/json e exatamente oito campos: schemaVersion=1, contract=PERSISTENCE_RECEIPT_V1, eventId, deliveryId, organizationId, endpointId, receiptId UUID estável e persistedAt UTC ISO. Correlacione organização/evento ao envelope e delivery/endpoint aos headers. 204, HTML, IDs divergentes ou schema inválido não confirmam esse modo; o mesmo retry retorna o recibo já persistido.

Até dez tentativas totais. Rede, 5xx, 408, 425 e 429 permitem retry limitado; outros códigos, incluindo redirect, são terminais. Recibo ausente/inválido recebido permite retry limitado. DELIVERED depende de ackMode: em HTTP é transporte 2xx; no modo de recibo é aceite de evidência correlacionada. Nenhum dos modos comprova processamento, entrega WhatsApp ou custódia de mídia.

## GET - polling e diagnóstico

```
GET /v1/partner-events?after=0&limit=50
GET /v1/partner-events?after=CURSOR_RETORNADO&limit=50
GET /v1/partner-webhook-deliveries?limit=50
```

Persista nextCursor como string decimal; não converta para Number no JavaScript. Retenção: 30 dias. 409 EVENT_CURSOR_EXPIRED fornece resetCursor/earliestCursor/lastPrunedCursor; reinicie e concilie com mensagens. Eventos históricos importados não são automaticamente novas mensagens acionáveis.

# Catálogo | conexões QR

Rotas e códigos extraídos do contrato público. {id} e demais parâmetros de caminho são obrigatórios; UUIDs do Broker, salvo IDs numéricos explicitamente descritos no controle Chatwoot. Nos exemplos JSON, UUIDs são sintéticos. Os schemas completos no Swagger definem tipos, limites e campos de resposta.

**`GET /v1/provider-accounts`**

**Consultar contas de provedor disponíveis**

Escopo: instances:read | HTTP: 200, 400, 401, 403, 404

Parâmetros: limit? [query]; cursor? [query]; provider? [query]

Lista as contas de provedor da sua organização.

**`GET /v1/instances`**

**Listar conexões da empresa**

Escopo: instances:read | HTTP: 200, 400, 401, 403, 404

Parâmetros: limit? [query]; cursor? [query]

Retorna uma página das instâncias do tenant.

**`POST /v1/instances`**

**Criar uma conexão**

Escopo: instances:write | HTTP: 200, 201, 202, 400, 401, 403, 404, 409, 415, 502

Parâmetros: idempotency-key* [header]

JSON (* obrigatório; ? opcional): name*, provider*, providerAccountId*

**`Exemplo: {"name":"Canal de teste","provider":"BAILEYS",`**
**`"providerAccountId":"11111111-1111-4111-8111-111111111111"}`**

**`GET /v1/instances/{id}`**

**Consultar uma conexão**

Escopo: instances:read | HTTP: 200, 400, 401, 403, 404

Retorna os dados da instância identificada por um UUID do Broker pertencente à empresa autenticada.

**`POST /v1/instances/{id}/connect`**

**Solicitar pareamento QR**

Escopo: instances:write | HTTP: 200, 202, 400, 401, 403, 404, 409, 415, 502

Parâmetros: idempotency-key* [header]

JSON (* obrigatório; ? opcional): pairingHint?

**`Exemplo: {}`**

**`GET /v1/instances/{id}/status`**

**Consultar estado de conexão**

Escopo: instances:read | HTTP: 200, 400, 401, 403, 404, 502

Observa o estado atual do provedor.

**`POST /v1/instances/{id}/disconnect`**

**Desconectar uma instância**

Escopo: instances:write | HTTP: 200, 202, 400, 401, 403, 404, 409, 415, 502

Parâmetros: idempotency-key* [header]

JSON (* obrigatório; ? opcional): {}

**`Exemplo: {}`**

Criar uma instância não concede controle Chatwoot. /connect também serve à reconexão; não existe /reconnect neste catálogo. Resultado pending/operationId exige acompanhamento. QR exibido, HTTP de sucesso e container saudável não comprovam entrega.

PDF, Swagger e imagens devem corresponder à mesma revisão. Código e documentação locais não comprovam implantação, migration aplicada ou mídia entregue/reproduzida em provider/host. Valide esses resultados no ambiente autorizado.

# Catálogo | mensageria e arquivos

**`POST /v1/messaging/instances/{id}/partner-activate`**
**Ativar mensageria para uma instância QR**
Escopo: instances:write + messaging:send | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 502, 503
Parâmetros: idempotency-key* [header]
JSON (* obrigatório; ? opcional): {}
**`Exemplo: {}`**

**`GET /v1/messaging/channels`**
**Listar canais de mensageria**
Escopo: messaging:read | HTTP: 200
Retorna os canais visíveis ao tenant.

**`GET /v1/messaging/channels/{id}/conversations`**
**Listar conversas de um canal**
Escopo: messaging:read | HTTP: 200
Consulta as conversas registradas no Broker para o canal.

**`GET /v1/messaging/conversations/{id}`**
**Consultar uma conversa**
Escopo: messaging:read | HTTP: 200
Retorna os dados da conversa canônica.

**`GET /v1/messaging/conversations/{id}/messages`**
**Consultar mensagens e estados**
Escopo: messaging:read | HTTP: 200
Retorna as mensagens registradas e seus estados observados.

**`GET /v1/messaging/media/{id}`**
**Baixar anexo privado registrado no Broker**
Escopo: messaging:read | HTTP: 200
Devolve bytes do anexo com seu MIME e Content-Disposition attachment.

**`POST /v1/messaging/channels/{id}/partner-text`**
**Enfileirar texto individual pelo QR**
Escopo: messaging:send | HTTP: 202, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503
Parâmetros: idempotency-key* [header]
JSON (* obrigatório; ? opcional): conversationId*, text* OU phoneNumber*, text*
**`Exemplo: {"conversationId":"11111111-1111-4111-8111-111111111111","text":"Mensagem de teste"}`**

**`POST /v1/messaging/channels/{id}/media`**
**Carregar anexo privado no canal QR**
Escopo: media:upload | HTTP: 201, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503
Parâmetros: Idempotency-Key* [header]; X-JRC-Media-Mime* [header]; X-JRC-Media-Kind* [header]; X-JRC-Media-Name* [header]
Corpo: application/octet-stream (bytes do arquivo).

**`GET /v1/messaging/channels/{id}/media/{mediaId}`**
**Consultar estado do anexo privado**
Escopo: media:upload | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503
Consulta a referência de upload no mesmo tenant/canal; não envia mensagem, não cria arquivo e não repete PUT.

**`POST /v1/messaging/channels/{id}/partner-media`**
**Enfileirar anexo com legenda pelo QR**
Escopo: messaging:send + media:send | HTTP: 202, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503
Parâmetros: idempotency-key* [header]
JSON (* obrigatório; ? opcional): caption?, conversationId*, mediaId* OU caption?, mediaId*, phoneNumber*
**`Exemplo: {"conversationId":"11111111-1111-4111-8111-111111111111",`**
**`"mediaId":"11111111-1111-4111-8111-111111111111"}`**

# Catálogo | eventos e erros

**`GET /v1/partner-webhooks`**
**Listar endpoints receptores**
Escopo: webhooks:manage | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503
Retorna os endpoints cadastrados e ackMode, sem seus segredos de assinatura.

**`POST /v1/partner-webhooks`**
**Cadastrar receptor HTTPS e negociar ACK**
Escopo: webhooks:manage | HTTP: 201, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503
Parâmetros: idempotency-key* [header]
JSON (* obrigatório; ? opcional): ackMode?, url*
**`Exemplo: {"url":"https://backend.example/hooks/broker"}`**

**`DELETE /v1/partner-webhooks/{id}`**
**Revogar receptor e cancelar entregas pendentes**
Escopo: webhooks:manage | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503
Revoga o endpoint, remove seu segredo e cancela trabalho pendente.

**`GET /v1/partner-events`**
**Consultar eventos por cursor**
Escopo: webhooks:manage | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503
Parâmetros: after? [query]; limit? [query]
Use after=0 na primeira leitura e depois o nextCursor retornado.

**`GET /v1/partner-webhook-deliveries`**
**Consultar entregas e evidências do ACK**
Escopo: webhooks:manage | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503
Parâmetros: limit? [query]
Lista até 100 entregas com estado, tentativas, ackMode e persistenceReceipt anulável.

O diagnóstico informa ackMode e persistenceReceipt anulável: schemaVersion, receiptId, persistedAt do receptor e acceptedAt do Broker. Evidência é gravada junto com DELIVERED sob lease válido; callback ou conclusão tardia não substitui esse contrato. Corpos remotos e segredos não são retornados.

Revogar o receptor cancela entregas pendentes e remove seu segredo; uma requisição já despachada pode chegar. Revogar a API key não revoga automaticamente o receptor.

## Tratamento de falhas e resultados incertos

| HTTP / estado | Ação do cliente |
|---|---|
| 400 / 422 | Corrigir parâmetros/corpo conforme schema. |
| 401 / 403 | Conferir credencial única, validade, escopos e vínculo. |
| 404 | Conferir recurso do tenant e disponibilidade do perfil. |
| 409 | Conciliar revisão, identidade, idempotência ou cursor. |
| 429 | Aplicar backoff, preservando a intenção idempotente. |
| 502 / 503 / timeout / UNKNOWN | Consultar a intenção existente. Não forçar reenvio com nova chave. |

Falhas estruturadas usam application/problem+json com code e requestId quando declarado pela rota. Preserve requestId nos logs sem incluir chaves, segredos ou payloads de clientes. Operações legadas podem declarar somente sucesso. A ausência de evento não comprova falha ou sucesso.

# Catálogo | controle Chatwoot (1/2)

Perfil opcional com chave de controle vinculada à Account autorizada, separado da chave genérica. X-JRC-External-Actor é atribuição auditável e não autentica o usuário remoto; campanhas exigem o ID numérico do administrador e revisões atuais. O backend consumidor revalida sua sessão e permissões locais.

**`GET /v1/integrations/chatwoot/control/context`**

**Consultar contexto de controle Chatwoot**

Escopo: chatwoot:read | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Perfil opcional vinculado a uma Account Chatwoot remota.

**`GET /v1/integrations/chatwoot/control/resources`**

**Listar recursos para vínculo Chatwoot**

Escopo: chatwoot:manage | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Obtém recursos autorizados para o assistente de conexão, incluindo instâncias e escolhas locais/remotas disponíveis.

**`GET /v1/integrations/chatwoot/control/onboarding`**

**Consultar operações de vínculo Chatwoot**

Escopo: chatwoot:manage | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Lista até 50 operações persistidas no contexto autorizado da Account, incluindo estado, etapa e IDs disponíveis.

**`POST /v1/integrations/chatwoot/control/onboarding`**

**Iniciar vínculo de caixa e conexão**

Escopo: chatwoot:manage | HTTP: 202, 400, 401, 403, 404, 409, 413, 415, 502, 503

Parâmetros: idempotency-key* [header]; X-JRC-External-Actor? [header]

JSON (* obrigatório; ? opcional): agentIds*, inboxId?, name*, replaceExistingWebhook*, source*

**`Exemplo: {"name":"Canal de teste","source":{"kind":"EXISTING",`**
**`"instanceId":"11111111-1111-4111-8111-111111111111"},"agentIds":[1],`**
**`"replaceExistingWebhook":false}`**

**`GET /v1/integrations/chatwoot/control/onboarding/{operationId}`**

**Consultar operação de vínculo**

Escopo: chatwoot:read | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Consulta os passos e o resultado da operação de onboarding existente, sem repetir a criação.

**`POST /v1/integrations/chatwoot/control/onboarding/{operationId}/recover`**

**Recuperar operação de vínculo**

Escopo: chatwoot:manage | HTTP: 202, 400, 401, 403, 404, 409, 413, 415, 502, 503

Parâmetros: idempotency-key* [header]; X-JRC-External-Actor? [header]

JSON (* obrigatório; ? opcional): action*

**`Exemplo: {"action":"RETRY"}`**

**`GET /v1/integrations/chatwoot/control/connections/{integrationId}/status`**

**Consultar saúde da conexão integrada**

Escopo: chatwoot:read | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Observa sessão, identityApproved, identityStatus, identityRevision, número mascarado, callback, evidências e allowedActions.

**`POST /v1/integrations/chatwoot/control/connections/{integrationId}/pair`**

**Solicitar QR da conexão integrada**

Escopo: chatwoot:pair; primeira identidade exige também chatwoot:manage | HTTP: 200, 202, 400, 401, 403, 404, 409, 413, 415, 502, 503

Parâmetros: idempotency-key* [header]; X-JRC-External-Actor? [header]

JSON (* obrigatório; ? opcional): {}

**`Exemplo: {}`**

# Catálogo | controle Chatwoot (2/2)

**`GET /v1/integrations/chatwoot/control/connections/{integrationId}/pair-operations/{operationId}`**

**Consultar operação de pareamento**

Escopo: chatwoot:pair; identidade ainda não aprovada exige também chatwoot:manage | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Obtém progresso durável sem iniciar outra operação nem aprovar identidade.

**`POST /v1/integrations/chatwoot/control/connections/{integrationId}/confirm-identity`**

**Confirmar identidade observada do QR**

Escopo: chatwoot:manage | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Parâmetros: idempotency-key* [header]; X-JRC-External-Actor? [header]

JSON (* obrigatório; ? opcional): observedRevision*

**`Exemplo: {"observedRevision":1}`**

**`POST /v1/integrations/chatwoot/control/connections/{integrationId}/disconnect`**

**Desconectar número da integração**

Escopo: chatwoot:disconnect | HTTP: 200, 202, 400, 401, 403, 404, 409, 413, 415, 502, 503

Parâmetros: idempotency-key* [header]; X-JRC-External-Actor? [header]

JSON (* obrigatório; ? opcional): {}

**`Exemplo: {}`**

**`PUT /v1/integrations/chatwoot/control/connections/{integrationId}/agents`**

**Definir agentes da caixa integrada**

Escopo: chatwoot:manage | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Parâmetros: idempotency-key* [header]; X-JRC-External-Actor? [header]

JSON (* obrigatório; ? opcional): agentIds*

**`Exemplo: {"agentIds":[1]}`**

**`GET /v1/integrations/chatwoot/control/connections/{integrationId}/campaign-transport`**

**Consultar capacidade real do remetente para campanhas**

Escopo: chatwoot:read | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Retorna protocolo chatwoot-campaigns/v1, vínculo, revisões, identidade, ready e reason.

**`POST /v1/integrations/chatwoot/control/connections/{integrationId}/campaign-intents`**

**Reservar intenção canônica de campanha**

Escopo: chatwoot:manage | HTTP: 200, 201, 400, 401, 403, 404, 409, 413, 415, 502, 503

Parâmetros: idempotency-key* [header]; x-jrc-external-actor* [header]

JSON (* obrigatório; ? opcional): campaignId*, conversationId*, destinationRevision*, identityRevision*, kind*, messageId*, ownerRevision*, text*

```
Exemplo: {"conversationId":1,"messageId":1,"campaignId":"campanha-teste","kind":"text",
 "text":"Mensagem de teste","destinationRevision":1,"identityRevision":1,
 "ownerRevision":1}
```

**`GET /v1/integrations/chatwoot/control/connections/{integrationId}/conversations/{conversationId}/campaign-messages/{messageId}`**

**Consultar recibo agregado da campanha**

Escopo: chatwoot:read | HTTP: 200, 400, 401, 403, 404, 409, 413, 415, 502, 503

Usa IDs numéricos da conversa e mensagem da central vinculada, não UUIDs de conversa JRC.

## Identidade, agentes e campanhas

PUT /agents define o conjunto de agentIds da Account vinculada. GET status nunca aprova identidade. Somente vínculo QR novo elegível e uma intenção administrativa válida permitem matrícula inicial; legado ou troca de número exigem confirm-identity com observedRevision atual e decisão explícita. Não confirme automaticamente após erro.

READY/SUCCEEDED comprovam configuração; CONNECTED observa sessão; OPERATIONAL exige identidade e evidências atuais. Uma reserva de campanha exige host compatível e callback canônico único; não duplique pelo envio genérico. Este perfil não publica Flow e não certifica instalação externa.

## Versão e rastreabilidade

37 operações. Edição 2026-11-02. Contrato SHA-256:

**`bf01ed561085e6b71d1b45b6166087fa80ead6ee6972003944bb890076fe1408`**